Personalized learning with ChatGPT

Saturday 8th June, 2024 - 16:11

Hedi Tebourbi
University of Luxembourg
Email: hedi.tebourbi.001@student.uni.lu

This report has been produced under the supervision of:
Sana Nouzri
University of Luxembourg
Email: sana.nouzri@uni.lu

Abstract

This report presents the outcomes of Hedi Tebourbi's Bachelor Semester Project (BSP) supervised by Sana Nouzri. The project investigates the enhancement of personalized learning through autonomous AI agents within a custom tutoring platform. The scientific deliverable provides a comprehensive overview of personalized learning, adaptive learning techniques, and agent-oriented programming (AOP). It explains how these concepts integrate to create tailored educational experiences, backed by key literature emphasizing the transformative potential of AI in education.

The technical deliverable focuses on developing a multi-agent system using the CrewAI framework, implementing backend services with Django and SQLite, and creating a user-friendly frontend with React. The project demonstrates that integrating AOP and AI significantly enhances student engagement, motivation, and learning outcomes, providing a flexible and scalable educational environment tailored to individual learner needs. This work lays the groundwork for future advancements in AI-driven personalized learning systems, showcasing their potential to revolutionize traditional educational methods.

1. Introduction

The pursuit of personalized education has been a long-standing goal, aimed at catering to the unique needs and learning styles of each student. Recent advancements in artificial intelligence, particularly in natural language processing, have opened new avenues for achieving this goal. Autonomous AI agents, capable of adaptive learning and interactive dialogue, offer promising potential to revolutionize personalized education.

This project focuses on the development of a personalized tutoring platform that employs autonomous AI agents to enhance the learning experiences of students. The platform integrates the CrewAI framework with advanced language models to simulate tutor-student interactions, providing tailored exercises, detailed summaries, and clear explanations. By adaptively adjusting the complexity of tasks based on the student's performance, the system aims to ensure a paced and effective learning journey.

The platform leverages a multi-agent system where each agent has a specific role, including content generation, quiz creation, feedback analysis, and performance profiling. This comprehensive approach not only emulates the tutor-student interaction but also addresses various aspects of the learning process, from content delivery to assessment and feedback.

2. Project description

2.1. Domains

2.1.1. Scientific.

- Personalized Learning

  Personalized learning aims to tailor education to meet the individual needs of each student. This approach leverages data and technology to provide customized educational experiences that adapt to the learner's pace, preferences, and progress. By focusing on the unique strengths and weaknesses of each student, personalized learning seeks to enhance engagement, understanding, and retention. The integration of AI agents in this context allows for dynamic adjustments to educational content and methods, ensuring that each learner receives the support they need to succeed.

- Agent-Oriented Programming

  Agent-oriented programming (AOP) is a programming paradigm that focuses on creating and managing agents. In this context, an agent refers to a software entity that can independently perform actions, communicate with other agents or humans, and exhibit proactive behavior to achieve its goals. AOP offers a high-level abstraction for the design and implementation of complex systems, conceptualizing them as collections of interacting autonomous agents. The primary benefit of AOP is its capacity to encapsulate system complexities, thereby simplifying the processes of comprehension, development, and maintenance.

2.1.2. Technical.

- CrewAI Framework

  CrewAI is a multi-agent system framework that enables the development of applications involving multiple interacting agents. This framework supports the integration of various tools and models necessary for building a comprehensive tutoring platform.

- Advanced Language Models

  Advanced language models, such as OpenAI's ChatGPT, are utilized to understand and generate human-like text. These models facilitate natural and engaging interactions between the AI agents and students, making the learning experience more intuitive and effective.

- Django Framework

  Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It provides a robust and scalable platform for developing web applications with a focus on reusability of components.

- React Framework

  React is a JavaScript library for building user interfaces, particularly single-page applications where data changes over time. It allows developers to create large web applications that can update and render efficiently in response to data changes.

2.2. Targeted Deliverables

2.2.1. Scientific deliverables.

The scientific deliverable of this project aims to explore how autonomous AI agents can enhance personalized learning experiences. Key objectives include:

- Exploration of Personalized and Adaptive Learning Concepts: Examining the principles and benefits of personalized learning and how it can be effectively implemented through adaptive techniques using AI.

- Application of Agent-Oriented Programming (AOP): Investigating the AOP paradigm and its suitability for modeling and simulating complex educational systems.

- Integration of Artificial Intelligence (AI) in Education: Analyzing the role of AI in personalized tutoring, focusing on adaptive learning paths, intelligent tutoring systems, and data-driven insights.

By achieving these objectives, the project will demonstrate how a combination of AOP and AI can provide a robust framework for personalized learning, enhancing the educational experiences for students.

2.2.2. Technical deliverables.

The technical deliverable focuses on the development and implementation of the personalized tutoring platform, specifically:

- Backend Development:
  - Developing the backend using the Django framework.
  - Implementing an SQLite database for storing user information, learning materials, quiz results, and other relevant data.
  - Creating API endpoints to handle requests from the frontend and manage interactions between agents and users.
- Frontend Development:
  - Building the user interfaces using the React framework.
  - Creating various components to manage user interactions, display content, and handle state management.
  - Ensuring seamless communication between the frontend and backend through API calls for data retrieval and updates.
- Designing and Implementing the Multi-Agent System:
  - Utilizing the CrewAI framework to create a robust multi-agent system.
  - Implementing autonomous agents for various educational tasks, including content generation, quiz creation, feedback analysis, and performance profiling.

3. Pre-requisites

3.1. Scientific pre-requisites

No specific scientific prerequisites are mandatory to fully comprehend the scientific deliverable of the report.

3.2. Technical pre-requisites

Proficiency in Python, HTML, CSS and JavaScript programming languages.

4. Enhancing Personalized Learning with AI Agents

4.1. Requirements

The scientific deliverable aims to address the following question: **How do autonomous AI agents in a personalized tutoring platform enhance students' learning experiences?** To effectively address this question, the following functional and non-functional requirements were established:

Functional Requirements.

- Provide an overview of the fundamental concepts and methodologies involved in personalized learning with AI agents.
- Describe the core principles and techniques of adaptive learning.
- Establish a clear relationship between AI agent functionalities and personalized learning outcomes.

Non-functional Requirements.

- The scientific report must be well-structured and use accessible language.
- Concepts must be precise and accompanied by clear explanations.
- Illustrations and examples should be used to facilitate understanding.

4.2. Design

The scientific production is structured to present a comprehensive explanation of how autonomous AI agents can enhance personalized learning. The report is divided into four main sections, each building upon the previous to provide a logical progression from understanding personalized learning to exploring AI agent functionalities and their usage in this context.

4.2.1. Personalized Learning.

This section provides a comprehensive definition of personalized learning, explaining its key characteristics, benefits, and challenges. It establishes a clear understanding of the concept and its significance in modern education.

The primary source for understanding personalized learning is the article "Intelligent agent supported personalization for virtual learning environments" [1] exploring the significant improvements in eLearning effectiveness through intelligent agent technology. This article highlights the essential role of personalization in enhancing student engagement and learning outcomes by aligning educational content with individual learner needs.

In addition to the primary source, the article "Investigating AI-Powered Tutoring Systems that Adapt to Individual Student Needs, Providing Personalized Guidance and Assessments" [2] provides valuable insights into how AI-powered tutoring systems can revolutionize student learning by offering tailored guidance and assessments. This work underscores the transformative potential of AI in creating dynamic and responsive educational environments.

Another significant reference is "Personalized learning through AI," [3] which delves into how AI-driven personalized learning experiences tailor educational content and interactions to individual learners' unique needs, preferences, and pace. This publication emphasizes the adaptability and customization made possible by advanced AI techniques in personalized learning systems.

4.2.2. Adaptive Learning Techniques.

This section delves into the concept of adaptive learning, emphasizing its importance in tailoring educational support to individual students. It discusses various techniques used to adjust the difficulty and nature of tasks based on the student's performance and learning pace.

The primary reference for adaptive learning techniques is the article "Individualized AI Tutor Based on Developmental Learning Networks," [4] which demonstrates how an AI tutor application integrates developmental learning networks to efficiently support children's language learning. This article provides foundational knowledge on the implementation and benefits of adaptive learning systems.

Additional references that contribute to the understanding of adaptive learning include "iTutor: Promoting AI Guided Knowledge Interaction in Online Learning," [5] which highlights the role of AI-based conversational agents in enhancing interactive feedback and learner satisfaction in online education. This study underscores the importance of real-time feedback and interaction in adaptive learning environments.

Furthermore, "Development of an Autonomous, Intelligent and Adaptive E-learning System" [6] explores how adaptive e-learning systems can replicate the traditional human tutoring process by adjusting learning courses based on individual learner's abilities, thereby improving information literacy among adolescents. This reference is critical for understanding the practical applications and benefits of adaptive learning techniques.

4.2.3. Agent-oriented Programming (AOP).

This section focuses on the principles of agent-oriented programming and its significance in modeling complex systems such as personalized and tailored tutoring.

The primary reference for this section is the article "Agent-oriented programming" [7] which explains the AOP framework as an extension of object-oriented programming, emphasizing agents' mental states and their interactions through communication primitives. This foundational work provides a detailed explanation of the AOP paradigm and its relevance to complex system modeling.

In addition to the article, "Innovations in multi-agent systems" [8] provides insights into the next generation of agents focusing on human-centric interaction to achieve intelligence, utilizing a Belief, Desire, and Intention (BDI)

architecture. This publication is essential for understanding how AOP can be applied to create intelligent and responsive agents.

Another valuable reference is "Agent programming in the cognitive era," [9] which discusses BDI-based agent programming languages as an ideal framework for integrating AI capabilities into intelligent systems. This work highlights the flexibility and scalability of AOP in developing sophisticated AI-driven systems.

4.2.4. Integration of AOP and AI in Personalized Learning.

The integration of AOP and AI within a personalized learning platform creates a powerful framework for delivering customized educational experiences.

The primary source for this integration is the article "An Agent-Oriented Architecture For Strategy-Based Personalized E- Learning," [10] which details how AOP is used to model complex systems like personalized and tailored tutoring through an ontology-based information model. This reference provides a practical example of integrating AOP with AI to create effective educational systems.

Additional references that contribute to understanding this integration include "Personal Assistance Agent in Programming Tutoring System," [11] which explores how a personal assistance agent dynamically adapts educational material and the user interface to individual learners in the Protus system. This study is significant for its demonstration of adaptive and personalized content delivery through agent-oriented programming.

Lastly, "On agent-based software engineering" [12] discusses the broader implications of agent-based software in improving the modeling, designing, and implementation of computer systems, particularly in dynamic and uncertain environments. This reference is crucial for understanding the theoretical and practical benefits of AOP in creating robust and flexible AI-driven systems.

4.3. Production

This section will detail the implementation of the AOP concepts and AI within the personalized tutoring platform, demonstrating how they contribute to the overall functionality and effectiveness of the system.

4.3.1. Personalized Learning.

Personalized learning is an educational approach that focuses on tailoring education to the unique needs, preferences, and strengths of each student. This method leverages data and technology to create customized learning paths that adapt to an individual's pace, style, and progress. Key characteristics of personalized learning include the customization of educational content and interactions to fit the unique requirements of each learner, as discussed in "Personalized learning through AI" [3].

Additionally, AI-powered tutoring systems, which adapt to individual student needs and provide personalized guidance and assessments, underscore the transformative potential of AI in education. This is highlighted in the article "Investigating AI-Powered Tutoring Systems" [2]. Intelligent agents also play a crucial role in enhancing eLearning effectiveness by aligning educational content with individual learner needs, as described in "Intelligent agent supported personalization for virtual learning environments" [1]. The benefits of personalized learning include increased student engagement, improved educational outcomes, and the flexibility for students to learn at their own pace, thereby reducing frustration and enhancing motivation.

However, the implementation of personalized learning systems can be complex and resource-intensive, with significant challenges in data privacy and ensuring equitable access to these educational tools.

Overall, personalized learning, powered by advanced AI techniques, creates dynamic and responsive educational environments that cater to the unique needs of each student, thereby making learning more engaging and effective while also presenting challenges that need to be addressed.

4.3.2. Adaptive Learning Techniques.

Adaptive learning is a critical component of personalized education, designed to dynamically tailor the learning experience to meet the individual needs and performance of each student. Unlike traditional educational methods that apply a one-size-fits-all approach, adaptive learning systems adjust the difficulty and nature of tasks based on the student's progress, learning pace, and demonstrated understanding. This approach ensures that students receive the right level of challenge and support, which is essential for effective learning and engagement. The article "Individualized AI Tutor Based on Developmental Learning Networks" [4] provides foundational knowledge on the implementation and benefits of such systems, highlighting how they can efficiently support learning by integrating developmental learning networks.

One of the core concepts of adaptive learning is the use of advanced AI algorithms to continuously assess a student's performance and adapt the educational content accordingly. These systems analyze various data points, such as quiz results, interaction patterns, and time spent on tasks, to identify each student's strengths and areas for improvement. By doing so, they can personalize the learning path, ensuring that students are neither bored by content that

is too easy nor overwhelmed by material that is too difficult. This concept is further elaborated in "iTutor: Promoting AI Guided Knowledge Interaction in Online Learning," [5] which discusses how AI-based conversational agents enhance interactive feedback and learner satisfaction in online education.

Real-time feedback is another essential element of adaptive learning. AI-powered tutoring systems provide immediate and interactive feedback, helping students understand their mistakes and learn from them on the spot. This instant feedback loop keeps students engaged and motivated, as they can see their progress and understand what they need to work on next. It also allows educators to monitor student progress more effectively and intervene when necessary.

Moreover, adaptive learning techniques replicate the benefits of one-on-one tutoring by providing personalized support that evolves with the student's needs. As the student progresses, the system can introduce more complex concepts and adjust the learning strategies to match their developing skills. This level of personalization is particularly beneficial for students who might struggle in a traditional classroom setting, as it ensures they receive the attention and resources they need to succeed. The "Development of an Autonomous, Intelligent and Adaptive E-learning System" [6] explores how these systems can replicate traditional tutoring processes, improving information literacy among learners.

In addition to improving individual learning outcomes, adaptive learning systems can enhance overall educational efficiency. By automating the personalization process, these systems free up educators to focus on more strategic tasks, such as providing additional support to students who need it most and developing new instructional materials.

4.3.3. Agent-Oriented Programming (AOP).

Agent-oriented programming (AOP) is a paradigm in computer science that models a system as a collection of autonomous entities, known as agents, which interact with each other and their environment to achieve specific goals. This approach is particularly significant in modeling complex systems, such as personalized and tailored tutoring environments, where agents can represent various entities and functionalities within the system.

One of the fundamental principles of AOP is the conceptualization of agents as entities with mental states, including beliefs, desires, and intentions. This idea, initially presented in the seminal work "Agent-oriented programming," [7] extends the principles of object-oriented programming by incorporating the mental state of agents and their interactions through communication primitives. This extension allows for more complex and dynamic interactions within the system, as agents can make decisions based on their mental states and the information they receive from other agents.

Another key principle of AOP is the use of the Belief-Desire-Intention (BDI) architecture, which is crucial for developing intelligent and responsive agents. The BDI model provides a framework for designing agents that can reason about their actions and make decisions based on their beliefs (information about the world), desires (goals they want to achieve), and intentions (plans to achieve those goals). This architecture is detailed in the article "Innovations in multi-agent systems," [8] which discusses the next generation of agents focusing on human-centric interactions to achieve intelligence.

The flexibility and scalability of AOP are also critical principles, enabling the development of sophisticated AI-driven systems that can adapt to changing environments and user needs. This adaptability is essential for integrating AI capabilities into intelligent systems, as discussed in "Agent programming in the cognitive era," [9] which highlights BDI-based agent programming languages as an ideal framework for such integration.

4.3.4. Integration of AOP and AI in Personalized Learning.

The integration of agent-oriented programming (AOP) and artificial intelligence (AI) within personalized learning platforms creates a powerful framework for delivering customized educational experiences. This approach leverages the strengths of both AOP and AI to create dynamic, adaptive, and highly personalized learning environments that cater to the unique needs of each student.

The core idea behind this integration is to use AOP to model complex educational systems as collections of autonomous agents. Each agent represents different aspects of the learning environment, such as the tutor, the learning materials, and the student profile. This agent-based approach allows the system to simulate interactions similar to those in human tutoring, adapting in real-time to the needs and progress of the student. The article "An Agent-Oriented Architecture For Strategy-Based Personalized E-Learning" [10] provides a detailed example of how AOP can be used to model personalized tutoring systems through an ontology-based information model, demonstrating the practical application of these principles.

AI enhances this framework by enabling the agents to make intelligent decisions and provide personalized recommendations based on data analysis and machine learning algorithms. For instance, a personal assistance agent can dynamically adapt educational material to fit the individual learner's needs. This capability is highlighted in the study "Personal Assistance Agent in Programming Tutoring System," [11] which explores the use of personal

assistance agents in the Protus system[1] to deliver adaptive and personalized content.

Furthermore, the integration of AOP and AI supports the development of robust and flexible educational systems capable of operating in dynamic and uncertain environments. The theoretical and practical benefits of this integration are discussed in the article "On agent-based software engineering," [12] which emphasizes the improvements in modeling, designing, and implementing complex computer systems using agent-based approaches.

4.4. Assessment

The research aimed to address the question: "How do autonomous AI agents in a personalized tutoring platform enhance students' learning experiences?" To effectively evaluate this question, the assessment focused on several key aspects.

4.4.1. Overview of Personalized Learning with AI Agents. The scientific deliverable provided an in-depth overview of the fundamental concepts and methodologies involved in personalized learning with AI agents. The primary sources, such as "Intelligent agent supported personalization for virtual learning environments" [1] and "Investigating AI-Powered Tutoring Systems," [2] effectively demonstrated the transformative potential of AI in education. These references highlighted how AI-driven systems can tailor educational content and interactions to individual learner needs, significantly improving student engagement and learning outcomes. This aligns well with the functional requirement of providing an overview of personalized learning with AI agents.

4.4.2. Core Principles and Techniques of Adaptive Learning. The deliverable thoroughly explored the core principles and techniques of adaptive learning. It discussed various methods for adjusting the difficulty and nature of educational tasks based on student performance and learning pace. Key references like "Individualized AI Tutor Based on Developmental Learning Networks" [4] and "iTutor: Promoting AI Guided Knowledge Interaction in Online Learning" [5] provided foundational knowledge on the implementation and benefits of adaptive learning systems. These studies emphasized the importance of real-time feedback and interaction, aligning with the requirement to describe the core principles and techniques of adaptive learning.

4.4.3. Relationship Between AI Agent Functionalities and Personalized Learning Outcomes. A clear relationship between AI agent functionalities and personalized learning outcomes was established. The research showed how AI-driven systems adjust learning paths based on individual performance, resulting in improved educational outcomes. For example, the articles "Personalized learning through AI" [3] and "Development of an Autonomous, Intelligent and Adaptive E-learning System" [6] illustrated how AI-driven systems personalize learning experiences, leading to better engagement and educational efficiency. The integration of agent-oriented programming (AOP) and AI within personalized learning platforms, as discussed in "An Agent-Oriented Architecture For Strategy-Based Personalized E-Learning," [10] demonstrated how AOP can enhance the adaptability and responsiveness of educational systems. This effectively meets the requirement of linking AI functionalities to personalized learning outcomes.

4.4.4. Structure and Accessibility. The scientific report was well-structured, facilitating a logical progression from understanding personalized learning to exploring AI agent functionalities and their applications in personalized tutoring. The clear division into sections and the use of accessible language ensured that the content was understandable to a broad audience, fulfilling the non-functional requirement of using accessible language.

4.4.5. Precision and Clarity. Concepts were presented with precision and clarity, accompanied by detailed explanations and supporting references. The use of primary and additional sources provided a robust framework for understanding the various aspects of personalized and adaptive learning. Illustrations and examples, such as those found in "Personal Assistance Agent in Programming Tutoring System," [11] were effectively used to facilitate understanding. This meets the non-functional requirement of providing clear explanations and illustrative examples.

4.4.6. Strengths and Limitations of AI in Personalized Learning. The research assessed the strengths and limitations of AI and AOP as a modeling paradigm for personalized learning. It highlighted the benefits of personalized learning, such as increased student engagement and improved educational outcomes, while also acknowledging the challenges, such as data privacy concerns and resource-intensive implementation. The flexibility and scalability of AOP in developing sophisticated AI-driven systems were also emphasized.

4.4.7. Insights and Future Research. The research provided valuable insights into the applicability of AI and AOP in personalized learning platforms. Despite the simplifications and approximations made, the findings offer a comprehensive understanding of the strengths and limitations of AI in this context. For future research, it would be beneficial to explore more complex scenarios of personalized learning, incorporating additional factors such as social structures, and cultural dynamics. Addressing these complexities can lead to a more comprehensive understanding of the strengths and limitations of AI and AOP in modeling personalized learning environments.

1. An adaptive educational platform developed as part of the research [11]. It uses AI and agent-oriented programming to personalize tutoring by dynamically adapting educational materials and the user interface to individual learner needs.

4.4.8. Conclusion. The research effectively addressed both the functional and non-functional requirements. The comprehensive overview of personalized learning, detailed explanation of adaptive learning techniques, and clear establishment of the relationship between AI functionalities and learning outcomes were well-aligned with the functional requirements. Additionally, the well-structured, precise, and accessible presentation, supported by relevant illustrations and examples, met the non-functional requirements. Overall, the research demonstrated how autonomous AI agents enhance students' learning experiences, providing a robust framework for future studies in this field.

5. Development and Implementation of Tutor AI

5.1. Requirements

The technical deliverable focuses on the development of a personalized tutoring platform powered by autonomous AI agents. The key requirements are categorized into three blocks: Backend Development, Frontend Development, and Multi-Agent System and Natural Language Processing.

5.1.1. Backend Development.

- Django Framework [13]: Use the Django Framework for backend development, handling authentication, data management, and API endpoints[2].
- SQLite Database [14]: Implement SQLite as the built-in database to store user information, learning materials, quiz results, and other relevant data.
- API Endpoints: Create endpoints to handle requests from the frontend and manage interactions between the agents and the user.

5.1.2. Frontend Development.

- React Framework [15]: Use the React Framework for building user interfaces, ensuring an intuitive and user-friendly experience.
- Components: Develop various React components to handle user interactions, display content, and manage state.
- Integration: Facilitate communication between the frontend and backend through API calls, ensuring data retrieval and updates.

5.1.3. Multi-Agent System and Natural Language Processing.

- Multi-Agent System: Design and implement a multi-agent system using the CrewAI framework [16] to support personalized learning experiences.
- Natural Language Processing (NLP): Integrate advanced language models, such as OpenAI's ChatGPT [19], to enable natural language interactions and content generation.
- PDF Processing: Implement a tool to convert uploaded PDF lessons into text format for further processing.
- Content Generation: Develop functionalities for generating customized learning materials in LaTeX format and compiling them into PDF documents.
- Quiz Creation and Correction: Implement tasks for creating quizzes in JSON[3] format and providing detailed feedback on quiz results.
- Student Profiling: Analyze student preferences and adapt the learning approach accordingly.
- Feedback Analysis: Refine educational content based on student interactions and feedback.
- Report Generation: Validate content and generate comprehensive reports for teachers.
- Chatbot Interaction: Facilitate natural language interactions with students through a communication chatbot.

5.2. Design

The design section offers a comprehensive overview of the structure and architecture of the personalized tutoring platform, detailing the backend and frontend components, as well as the multi-agent system.

5.2.1. Backend Development.

The backend development of the personalized tutoring platform involves using the Django framework to handle the core functionality, including authentication, data management, and API endpoints. The design focuses on ensuring a robust and scalable backend that supports the multi-agent system and interacts seamlessly with the frontend.

- **Django Framework:** Django is chosen for its comprehensive features and ease of use in developing complex web applications. It handles the core functionality, including:
  - Authentication: Managing user registration, login, and secure access to the platform.
  - Data Management: Handling the storage and retrieval of user data, learning materials, quiz results, and feedback.
  - API Endpoints: Creating RESTful API [22] endpoints to facilitate communication between the frontend and backend, and manage interactions between the agents and the user.

- **SQLite Database:** The built-in SQLite database is chosen for its simplicity and seamless integration with Django. The database design includes the classes and their relationships represented in the class diagram below:

2. Specific URLs within an Application Programming Interface (API) that define where resources can be accessed or actions can be performed.

3. JavaScript Object Notation [21] is a lightweight data interchange format that is easy for humans to read and write and easy for machines to parse and generate.

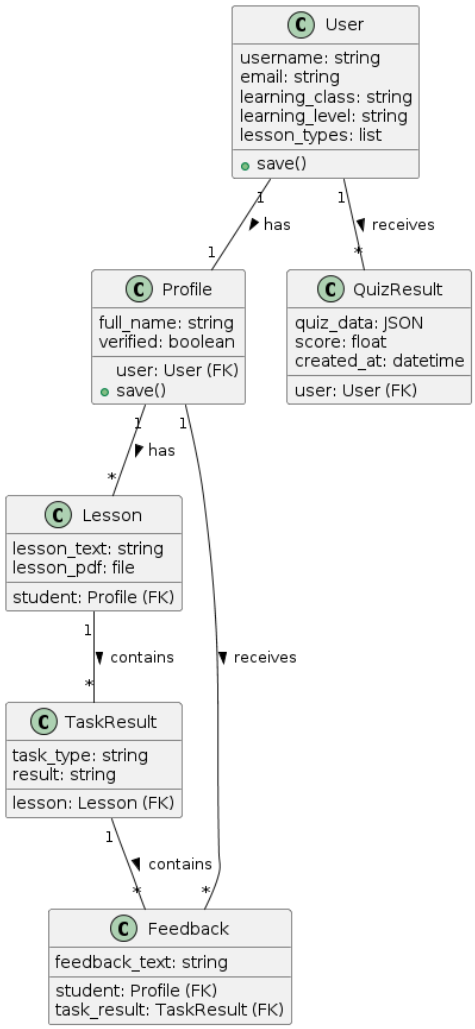


Fig. 1: Class Diagram of the Database

- User Information: Stores basic information about users, such as their username, email, learning class, learning level, and lesson types.
- Profile: Extends the User class and includes additional attributes like full name and verification status.
- Lesson: Represents the lessons uploaded by students. It can contain either text or PDF versions.
- Task Result: Stores the results of tasks executed by various agents.
- Feedback: Contains feedback provided by students on various tasks.
- Quiz Result: Stores quiz results, including the data, score, and timestamp of creation.

5.2.2. Frontend Development.

The frontend development focuses on developing a responsive and user-friendly interface using the React framework. The design ensures that students can easily interact with the platform, access learning materials, take quizzes, and communicate with the AI agents.

React Framework: React is chosen for its component-based architecture, which allows for the creation of reusable User Interface (UI) components and efficient state management. The key aspects of the frontend development include:

- **Components:** Various React components are developed to handle user interactions, display content, and manage state. These include:
  - Registration and Login: Components for user registration and login, including form validation and error handling.
  - Lesson Upload and Display: Components for uploading lessons, displaying text and PDF content, and managing the lesson state.
  - Flashcards Page: Components for displaying and interacting with flashcards, helping students review key concepts and information.
  - Quiz Interaction: Components for displaying quizzes, capturing user responses, and submitting results.
  - Chatbot Interface: Components for interacting with the communication chatbot, including text input and message display.

- **User Experience:** Ensuring a seamless and responsive user experience by incorporating:
  - Loading States: Providing feedback to users during data retrieval and processing.
  - Error Handling: Displaying appropriate error messages and handling exceptions gracefully.

5.2.3. Multi-Agent System and Natural Language Processing.

The Multi-Agent System (MAS) within our personalized learning platform leverages the CrewAI framework to manage various AI agents, each with specific roles and interactions. This design ensures the system can deliver customized educational experiences by coordinating the efforts of multiple specialized agents.

- **Agent Roles and Responsibilities**
  The system comprises agents with well-defined roles and responsibilities, critical for the platform's functionality. The agents in our system include:
  - *Learning Path Tutor:* Generates customized learning materials based on student preferences and feedback.
  - *Practical Tutor:* Creates quizzes and provides feedback on quiz results.
  - *Profiler:* Analyzes student preferences and learning progress.

- *Digital Teacher:* Validates content and generates comprehensive reports for teachers.
- *Communication Chatbot:* Interacts with students and answers their questions.

- **Interaction and Communication**
  Effective communication between agents is ensured through built-in mechanisms for task delegation and result sharing. This enhances collaboration, leading to a more cohesive learning experience.

- **Process Management**
  Task execution is supported by various processes, focusing on sequential execution to ensure structured educational activities. The sequential process allows tasks to be executed one after the other in a predefined order, providing a clear and logical progression for educational activities.

  Additionally, the framework supports asynchronous task execution, enabling specific agents to perform tasks outside the main execution loop. This flexibility allows for real-time responses and interactions, enhancing the adaptability and responsiveness of the system.

- **Custom Tools**
  To handle specialized tasks, custom tools are integrated into the agents. These tools enable specific functionalities such as PDF processing and LaTeX compilation.

5.3. Production

The production of the personalized tutoring platform involved several key stages, each focusing on different components: the backend development using Django, the frontend development using React, and the multi-agent system with CrewAI. This section provides a detailed overview of how each component was built and integrated to create the final product.

5.3.1. Backend Development.

The backend of the personalized tutoring platform is developed using the Django framework, leveraging its built-in SQLite database for data management. This section details the implementation of various components, including user authentication, data models, API endpoints, and the integration of CrewAI for managing the multi-agent system.

- **Django Framework Setup**

  The backend production process involved setting up the Django project, defining data models, creating serializers and views, configuring URLs, and integrating CrewAI for managing the multi-agent system. Additionally, secure user authentication was implemented using JSON Web Tokens (JWT)[4], ensuring that only authorized users can access the platform. This comprehensive setup ensures efficient data management, secure user authentication, and effective communication between the agents and the user, providing a robust backend for the personalized tutoring platform.

- **Defining Data Models**

  The `models.py` file defines the database schema for the application using Django's Object-relational Mapping layer (ORM)[5]. Each class represents a table in the database, with attributes corresponding to table columns. This file includes models for `User`, `Profile`, `Lesson`, `TaskResult`, `Feedback`, and `QuizResult`.

  - User: Extends Django's `AbstractUser` to add additional fields such as `learning_class`, `learning_level`, and `lesson_types`.
  - Profile: Linked to the User model via a one-to-one relationship, storing additional user details like `full_name` and verified status.
  - Lesson: Represents a lesson, containing fields for text and PDF versions of the lesson content.
  - TaskResult: Stores results of tasks related to lessons, such as content generation or quiz creation.
  - Feedback: Stores feedback provided by students on specific tasks.
  - QuizResult: Stores quiz results, including the user, quiz data, score, and creation timestamp.

```python
from django.db import models
from django.db.models.signals import post_save
from django.contrib.auth.models import
    AbstractUser

class User(AbstractUser):
    username = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    learning_class = models.CharField(max_length
    =50, blank=True, null=True)
    learning_level = models.CharField(max_length
    =50, blank=True, null=True)
    lesson_types = models.JSONField(default=list
    , blank=True)  # Store lesson types as a
    list

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['username']

class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=
    models.CASCADE)
    full_name = models.CharField(max_length
    =1000)
    verified = models.BooleanField(default=False
    )

```

4. Compact, URL-safe tokens used for securely transmitting information between parties as a JSON object, often used for authentication and authorization [23].

5. A tool that allows developers to interact with the database using Python code instead of SQL queries, mapping database tables to Python classes.

```
    def save(self, *args, **kwargs):
        if self.full_name == "" or self.full_name == None:
            self.full_name = self.user.username
        super(Profile, self).save(*args, **kwargs)

def create_user_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)

def save_user_profile(sender, instance, **kwargs):
    instance.profile.save()

post_save.connect(create_user_profile, sender=User)
post_save.connect(save_user_profile, sender=User)

# Store the lesson
class Lesson(models.Model):
    student = models.ForeignKey('Profile', on_delete=models.CASCADE)
    lesson_text = models.TextField(null=True, blank=True)
    lesson_pdf = models.FileField(upload_to='lessons/', null=True, blank=True)

# Store the task result
class TaskResult(models.Model):
    lesson = models.ForeignKey('Lesson', on_delete=models.CASCADE)
    task_type = models.CharField(max_length=100)
    result = models.TextField()

# Store the feedback
class Feedback(models.Model):
    student = models.ForeignKey('Profile', on_delete=models.CASCADE)
    task_result = models.ForeignKey(TaskResult, on_delete=models.CASCADE)
    feedback_text = models.TextField()


# Store the quiz result
class QuizResult(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    quiz_data = models.JSONField()
    score = models.FloatField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.user.username} - {self.score}"

```

Listing 1: models.py

- **API Endpoints**

  The `serializer.py` file contains serializers that define how data is converted between complex types (like model instances) and native Python data types that can be easily rendered into JSON or other content types. It uses Django Rest Framework (DRF) to handle serialization and deserialization of data.

```
from api.models import User, Profile, Lesson, TaskResult, Feedback, QuizResult
from django.contrib.auth.password_validation import validate_password
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer
from rest_framework import serializers

# Serializes User model data.
class UserSerializer(serializers.ModelSerializer):

    class Meta:
        model = User
        fields = ('id', 'username', 'email', 'learning_class', 'learning_level', 'lesson_types')

# Extends the token serializer to include additional user information.
class MyTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)

        token['full_name'] = user.profile.full_name
        token['username'] = user.username
        token['email'] = user.email
        token['verified'] = user.profile.verified
        return token

# Handles user registration and validation.
class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(
        write_only=True, required=True, validators=[validate_password])
    password2 = serializers.CharField(write_only=True, required=True)
    learning_class = serializers.CharField(max_length=50, required=False)
    learning_level = serializers.CharField(max_length=50, required=False)
    lesson_types = serializers.JSONField(required=False)

    class Meta:
        model = User
        fields = ('email', 'username', 'learning_class', 'learning_level', 'lesson_types' ,'password', 'password2')

    def validate(self, attrs):
        if attrs['password'] != attrs['password2']:
            raise serializers.ValidationError(
                {"password": "Password fields didn't match."})

        return attrs

    def create(self, validated_data):
        user = User.objects.create(
            username=validated_data['username'],
            email=validated_data['email'],
            learning_class=validated_data.get('learning_class', ''),
            learning_level=validated_data.get('learning_level', ''),
```

```
            lesson_types=validated_data.get('lesson_types', [])
        )

        user.set_password(validated_data['password'])
        user.save()

        return user

    def update(self, instance, validated_data):
        instance.username = validated_data.get('username', instance.username)
        instance.email = validated_data.get('email', instance.email)
        instance.learning_class = validated_data.get('learning_class', instance.learning_class)
        instance.learning_level = validated_data.get('learning_level', instance.learning_level)
        instance.lesson_types = validated_data.get('lesson_types', instance.lesson_types)
        if 'password' in validated_data:
            instance.set_password(validated_data['password'])
        instance.save()
        return instance

# Serializes Profile model data.
class ProfileSerializer(serializers.ModelSerializer):

    class Meta:
        model = Profile
        fields = [ 'id',  'user',  'full_name' ]

    def __init__(self, *args, **kwargs):
        super(ProfileSerializer, self).__init__(*args, **kwargs)
        request = self.context.get('request')
        if request and request.method=='POST':
            self.Meta.depth = 0
        else:
            self.Meta.depth = 3


# Crew AI serializers

"""
Integrated the CrewAI framework to manage the multi-agent system, enabling functionalities such as lesson submission, task result retrieval, and feedback provision.
"""

class LessonSerializer(serializers.ModelSerializer):
    student = serializers.PrimaryKeyRelatedField(read_only=True)

    class Meta:
        model = Lesson
        fields = ['id', 'lesson_text', 'lesson_pdf', 'student']

class TaskResultSerializer(serializers.ModelSerializer):
    class Meta:
        model = TaskResult
        fields = ['id', 'lesson', 'task_type', 'result']

class FeedbackSerializer(serializers.ModelSerializer):
    class Meta:
        model = Feedback
        fields = ['id', 'student', 'task_result', 'feedback_text']

class QuizResultSerializer(serializers.ModelSerializer):
    class Meta:
        model = QuizResult
        fields = '__all__'
```

Listing 2: serializer.py

The `views.py` file contains the logic for handling incoming HTTP requests, performing necessary operations, and returning appropriate HTTP responses. This file includes a variety of views to handle user authentication, profile management, and interactions with the multi-agent system (CrewAI).

```
import os
import json
import logging
from django.shortcuts import render
from django.http import JsonResponse, HttpResponse
from django.core.files.storage import default_storage
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import AllowAny, IsAuthenticated
from rest_framework.response import Response
from rest_framework import status
from rest_framework.views import APIView
from rest_framework import generics
from rest_framework_simplejwt.views import TokenObtainPairView
from .models import User, Profile, Lesson, TaskResult, Feedback, QuizResult
from .serializer import MyTokenObtainPairSerializer, RegisterSerializer, ProfileSerializer, UserSerializer, LessonSerializer, TaskResultSerializer, FeedbackSerializer, QuizResultSerializer
from .crewai.crew_setup import TutorCrew
from django.conf import settings

logger = logging.getLogger(__name__)

# Handles user login and token generation.
class MyTokenObtainPairView(TokenObtainPairView):
    serializer_class = MyTokenObtainPairSerializer

# Manages user registration.
class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()
    permission_classes = (AllowAny,)
    serializer_class = RegisterSerializer

@api_view(['GET'])
def getRoutes(request):
    routes = [
        '/api/token/',
        '/api/register/',
        '/api/token/refresh/'
    ]
```

```
    return Response(routes)

# Simple test endpoint to verify API functionality.
@api_view(['GET', 'POST'])
@permission_classes([IsAuthenticated])
def testEndPoint(request):
    if request.method == 'GET':
        data = f"Congratulations {request.user}, your API just responded to GET request"
        return Response({'response': data}, status=status.HTTP_200_OK)
    elif request.method == 'POST':
        text = "Hello buddy"
        data = f'Congratulations your API just responded to POST request with text: {text}'
        return Response({'response': data}, status=status.HTTP_200_OK)
    return Response({}, status.HTTP_400_BAD_REQUEST)

# Retrieves and updates user profiles.
class ProfileDetail(generics.RetrieveUpdateAPIView):
    serializer_class = ProfileSerializer
    queryset = Profile.objects.all()
    permission_classes = [IsAuthenticated]

# Allows users to update their information.
class UserUpdateView(generics.RetrieveUpdateAPIView):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [IsAuthenticated]

    def get(self, request, *args, **kwargs):
        instance = self.get_object()
        serializer = self.get_serializer(instance)
        return Response(serializer.data)

    def update(self, request, *args, **kwargs):
        partial = kwargs.pop('partial', False)
        instance = self.get_object()
        serializer = self.get_serializer(instance, data=request.data, partial=partial)
        serializer.is_valid(raise_exception=True)
        self.perform_update(serializer)
        return Response(serializer.data)
```

Listing 3: views.py

- **User Authentication**

  The user authentication process is essential for securing the platform and ensuring that only authorized users can access and interact with the system. This section details the steps taken to implement user authentication using Django Rest Framework (DRF) and JWT.

- **Integrating CrewAI**

  Integrated the CrewAI framework to manage the multi-agent system, enabling functionalities such as lesson submission, task result retrieval, and feedback provision.

```
# Crew AI views

# Handles lesson submission and interacts with CrewAI to process the lesson.
class SubmitLessonView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        profile = request.user.profile
        lesson_data = request.data.copy()
        lesson_serializer = LessonSerializer(data=lesson_data)

        if lesson_serializer.is_valid():
            lesson = lesson_serializer.save(student=profile)

            tutor_crew = TutorCrew(user_id=profile.user.id, lesson_id=lesson.id)
            pdf_path = tutor_crew.run_all_tasks()

            task_results = TaskResult.objects.filter(lesson=lesson)
            serialized_results = TaskResultSerializer(task_results, many=True)

            print(f"Generated PDF Path: {pdf_path}")

            return Response({
                'task_results': serialized_results.data,
                'pdf_path': pdf_path,
                'lesson_id': lesson.id  # Make sure to return the lesson ID
            }, status=status.HTTP_201_CREATED)
        else:
            # Log the validation errors
            print("Validation Errors:", lesson_serializer.errors)
            return Response(lesson_serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Retrieves task results for a specific lesson.
class GetTaskResultsView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request, lesson_id):
        task_results = TaskResult.objects.filter(lesson_id=lesson_id)
        serializer = TaskResultSerializer(task_results, many=True)
        return Response(serializer.data)

# Allows users to provide feedback on tasks.
class ProvideFeedbackView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request, lesson_id):
        profile = request.user.profile
        lesson = Lesson.objects.get(id=lesson_id)
        feedback_data = request.data.get('feedback')
        agent_key = request.data.get('agent_key')
        task_type = request.data.get('task_type')

        tutor_crew = TutorCrew(user_id=profile.user.id, lesson_id=lesson.id)

```

```
        updated_result = tutor_crew.execute_agent_task(agent_key, task_type, feedback_data)

        task_result = TaskResult.objects.create(
            lesson=lesson,
            task_type=task_type,
            result=updated_result
        )

        feedback = Feedback.objects.create(
            student=profile,
            task_result=task_result,
            feedback_text=feedback_data
        )

        return Response(TaskResultSerializer(task_result).data, status=status.HTTP_200_OK)

# Fetches quiz data.
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def get_quiz_data(request):
    logger.info("get_quiz_data called")
    try:
        file_path = os.path.join('output_files', 'quiz.json')
        if not os.path.exists(file_path):
            logger.error(f"Quiz file not found: {file_path}")
            return Response({'error': 'Quiz data not found.'}, status=status.HTTP_404_NOT_FOUND)
        with open(file_path) as quiz_file:
            quiz_data = json.load(quiz_file)
        return JsonResponse(quiz_data, safe=False)
    except json.JSONDecodeError as e:
        logger.error(f"Error parsing quiz data: {e}")
        return Response({'error': 'Error parsing quiz data.'}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)

# Submits quiz results.
class SubmitQuizResultView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        user = request.user
        quiz_data = request.data.get('quiz_data')
        score = request.data.get('score')

        # Create a new QuizResult object
        quiz_result = QuizResult.objects.create(user=user, quiz_data=quiz_data, score=score)

        return Response({'message': 'Quiz results saved successfully!'}, status=status.HTTP_201_CREATED)

#  Retrieves quiz results.
class FetchQuizResultsView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        user = request.user
        quiz_results = QuizResult.objects.filter(user=user)
        serializer = QuizResultSerializer(quiz_results, many=True)
        return Response(serializer.data, status=status.HTTP_200_OK)

# Fetches flashcards data.
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def get_flashcards_data(request):
    logger.info("get_flashcards_data called")
    try:
        file_path = os.path.join('output_files', 'flashcards.json')
        if not os.path.exists(file_path):
            logger.error(f"Flashcards file not found: {file_path}")
            return Response({'error': 'Flashcards data not found.'}, status=status.HTTP_404_NOT_FOUND)
        with open(file_path) as flashcards_file:
            flashcards_data = json.load(flashcards_file)
        return JsonResponse(flashcards_data, safe=False)
    except json.JSONDecodeError as e:
        logger.error(f"Error parsing flashcards data: {e}")
        return Response({'error': 'Error parsing flashcards data.'}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)

# Manages interactions with the chatbot.
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def chatbot_response(request):
    logger.info("chatbot_response called")
    user_message = request.data.get('message')
    user_id = request.user.id
    lesson_id = request.data.get('lesson_id')
    agent_key = request.data.get('agent_key')
    task_type = request.data.get('task_type')
    quiz_results = request.data.get('quiz_results')

    if not user_message:
        return Response({'error': 'No message provided.'}, status=status.HTTP_400_BAD_REQUEST)
    if not lesson_id:
        return Response({'error': 'No lesson_id provided.'}, status=status.HTTP_400_BAD_REQUEST)

    # Retrieve user data
    user = request.user
    profile = user.profile

    try:
        # Create the TutorCrew instance for communication chatbot
        tutor_crew = TutorCrew(
            user_id=user_id,
            lesson_id=lesson_id,
            quiz_results=quiz_results  # Pass quiz results to the TutorCrew
        )
        # Execute the communication_chatbot task if no specific agent or task type is provided
        if not agent_key and not task_type:
            response = tutor_crew.execute_agent_task(
                agent_key="communication_chatbot",
                task_type=None,
                feedback=user_message
            )
```

```
        else:
            response = tutor_crew.execute_agent_task(
                agent_key=agent_key,
                task_type=task_type,
                feedback=user_message
            )

        return JsonResponse({'response': response}, safe=False)
    except TypeError as e:
        logger.error(f"TypeError during chatbot response: {e}")
        return Response({'error': f"TypeError: {str(e)}"}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)
    except Exception as e:
        logger.error(f"Error during chatbot response: {e}")
        return Response({'error': 'Error processing message.'}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)

```

Listing 4: Crew AI views in views.py

By coordinating these components, the backend ensures efficient data management, secure user authentication, and effective communication between the user and the multi-agent system. This setup forms the backbone of the personalized tutoring platform.

5.3.2. Frontend Development.

The frontend of the personalized tutoring platform is developed using the React framework. Here, we outline the various components created, their functionalities, and how they integrate with the backend and CrewAI.

- **Component Creation**

  Each component in the React application is designed to handle a specific part of the user interface. Below is a list of the main components and their purposes, with the corresponding screenshots available in the Appendix:

  - App Component: The root component that sets up routing and renders other components based on the current route. It acts as the entry point of the application.
  - Login Component 5: Manages user authentication, allowing users to log in to the platform. It collects user credentials and interacts with the backend to validate and generate an authentication token.
  - Register Component 3: Handles user registration, allowing new users to create an account. It collects user information, including learning preferences, and interacts with the backend to create a new user profile.
  - Account Management Component 12: Manages account settings, allowing users to edit their user information such as email, password, and preferences. It provides forms for user updates and sends the changes to the backend.
  - Lesson Component 6: Displays lesson content fetched from the backend, including text and PDF files. It allows users to view and interact with lesson materials.
  - Quiz Component 18: Manages quizzes by displaying questions to the user and submitting their answers to the backend. It fetches quiz data and displays results after submission.
  - Flashcards Component 22: Displays flashcards for quick learning and revision. It fetches flashcard data from the backend and presents it to the user in an interactive manner.
  - Chatbot Component 14: Provides a chat interface for interaction with the AI-driven tutoring agent, and allow users to submit feedback on lessons and quizzes. It facilitates natural language queries and responses, sending user messages to the backend and displaying responses.

- **Auth Logic and Context Management**

  The frontend manages authentication and state using several key files:

  - AuthContext.js: Provides authentication context for the application, managing login, logout, and token management. It stores user information and tokens in local storage and ensures that the authentication state is maintained across sessions.
  - PrivateRoute.js: Protects routes that require authentication. It checks if a user is authenticated before allowing access to certain routes, redirecting unauthenticated users to the login page.
  - useAxios.js: Sets up an Axios instance[6] for handling API requests, including token refresh logic. It intercepts requests to check token expiry and refreshes the token if necessary, ensuring that API requests are authenticated.

- **Integration with Backend and CrewAI**

  The frontend interacts with the backend and CrewAI to provide a seamless user experience. Here's how the frontend components communicate with the backend:

  - User Authentication: Components like Login and Register interact with the backend API to authenticate users and manage user sessions. The backend generates and returns JWT tokens, which are stored in local storage and used for subsequent authenti-

6. A customizable version of the Axios HTTP client [24], allowing you to set default configuration options like base URL, headers, and timeouts for making HTTP requests.

cated requests.

- Fetching and Displaying Data: Components such as Account Management, Lesson, Quiz, and Flashcards use API endpoints to fetch data from the backend. For example, the Lesson component retrieves lesson content from the backend, and the Quiz component fetches quiz questions.
- Submitting Data: Components like Quiz and Feedback collect user input and send it back to the backend via POST requests. This data includes quiz answers, feedback on lessons, and other user-generated content.
- Chatbot Interaction: The Chatbot component sends user messages to the backend, where they are processed by CrewAI. The responses from CrewAI are displayed back to the user in the chat interface.
- Real-time Updates and Asynchronous Processing: The frontend uses state management to handle real-time updates and asynchronous operations. When a user submits a quiz, the Quiz component updates the UI based on the results returned by the backend.
- File Handling: The Lesson component manages PDF files, allowing users to view lesson content directly within the app. The backend provides URLs for these files, which are rendered in the frontend.

By utilizing React's component-based architecture, the frontend of the personalized tutoring platform effectively manages user interactions, state updates, and data communication with the backend and CrewAI. This setup ensures a responsive and intuitive user experience, aligning with the educational goals of the platform.

5.3.3. Implementing the Multi-Agent System with CrewAI.

The multi-agent system is a cornerstone of the personalized tutoring platform, enabling tailored learning experiences through autonomous agents. This section details the creation of the agents, their respective tasks, the custom tools developed, as well as the crew setup for detailed execution.

- **Creation of Agents**

  Each agent in the CrewAI framework was designed with specific roles and responsibilities to support various aspects of personalized learning. The primary agents included the Learning Path Tutor, Practical Tutor, Profiler, Digital Teacher, and Communication Chatbot.

  To elucidate the implementation process, we will provide a detailed example of the Learning Path Tutor's definition. This example serves as a prototype, illustrating the consistent methodology applied to the development of all other agents, each configured with specific tasks, tools, and roles.

Example of the Learning Path Tutor: This agent is responsible for generating customized learning materials based on the student's preferences and feedback. It processes lesson content, scrapes relevant online resources, and compiles the final materials in LaTeX format.

The agent is implemented following the CrewAI starter_template [17] as follows:

```
from crewai import Agent
from langchain_openai import ChatOpenAI
from .tools.search_tools import SearchTools
from .tools.browser_tools import BrowserTools
from .tools.output_storage_tool import
    store_output, compile_latex

class TutorAgents:
    def __init__(self):
        self.OpenAIGPT4o = ChatOpenAI(model_name
    ="gpt-4o", temperature=0.7)

    def learning_path_tutor(self):
        return Agent(
            role="Learning Path Tutor",
            backstory=dedent("""I'm a professor
    and personal tutor, designed to construct
    and adapt learning paths tailored to
    students' evolving needs."""),
            goal=dedent("""Create personalized
    course and flashcards that adapt to students
    ' learning styles and preferences."""),
            tools=[
                SearchTools.search_internet,
                BrowserTools.
    scrape_and_summarize_website,
                store_output,
                compile_latex,
            ],
            allow_delegation=False,
            verbose=True,
            llm=self.OpenAIGPT4o,
        )

```

Listing 5: Learning Path Tutor from Agents.py

Here, the `Learning Path Tutor` agent is created using the `Agent` class from CrewAI. The agent is initialized with a specific role, backstory, goal, and a set of tools it can use to perform its tasks. Tools include internet search, web scraping, output storage, and LaTeX compilation.

Tasks are defined with a clear description and expected output, specifying which agent will execute them. For example:

```
from crewai import Task
from textwrap import dedent

class TeachingTasks:
    def __init__(self):
        self.tip_section = "If you do your BEST
    WORK, I'll give you a $10,000 commission!"

```

```
    def adapt_learning_path(self, agent, student_name, lesson, student_class, student_level, student_learning_preferences, feedback):
        return Task(
            description=dedent(f"""
**Task**: Fully Develop and Customize Learning Materials
**Description**: Create a comprehensive and engaging course based on the provided lesson and tailor it according to the student's learning preferences.
Then, develop detailed educational flashcards that supplement the course materials.
Both the course and flashcards should be fully developed and ready for use by the student.
**Expected Output**:
    - A detailed course in LaTex format. This should include sections, key points, and interactive elements tailored to enhance learning.
    - A complete set of flashcards in a JSON file with 'front' and 'back', where each flashcard includes a term and a definition related to the course content.
    - A compiled PDF of the LaTeX document.
**Actions**:
    - Develop the course material to be complete and ready-to-deliver not just an outline but filled with detailed, educational content.
    - Create flashcards that are directly usable for study and revision, emphasizing the key concepts of the lesson.
    - Create the files "compiled_materials.tex" and "flashcards.json"
    - Compile the LaTeX document into a PDF format for easy distribution and use.
**Feedback Mechanism**:
    - Provide the profiler with a sample of the course and flashcards to receive initial feedback on their alignment with learning preferences.
**Parameters**:
    - Student Name: {student_name}
    - Lesson: {lesson}
    - Student Class: {student_class}
    - Student Level: {student_level}
    - Student Learning Preferences: {student_learning_preferences}
    - Student Feedback: {feedback}
**Note**: Please do NOT put in the file "LaTeX content as shown above" or "content from above" or "LaTeX content from above", JUST DO IT.
This task involves both the creation of the course, the development of supplementary materials and compiling the LaTeX file into a PDF file with 'front' and 'back'.
Ensure all components are finalized and ready for student use.
{self.tip_section}
            """),
            expected_output="A detailed and complete course in PDF format containing the course and a complete set of flashcards in a JSON file.",
            agent=agent,
            actions=[
                lambda inputs: agent.tools[-1](inputs['compiled_materials'], 'compiled_materials.tex'),
                lambda inputs: agent.tools[-1](inputs['flashcards_content'], 'flashcards.json')
            ]
        )
```

Listing 6: Learning Path Tutor from Tasks.py

Here, the `adapt_learning_path` task is defined for the `Learning Path Tutor` agent, specifying the task's description, expected output, and the actions required to complete the task.

- **Breakdown of Task Definition**

  - Description: The task description outlines the overall goal.
  - Expected Output: This section specifies the expected deliverables from the task.
  - Actions: The actions required to complete the task are defined using lambda functions. These actions leverage the tools provided to the agent.
  - Feedback Mechanism: A feedback mechanism is included to ensure the materials are aligned with the student's learning preferences.
  - Parameters: The task parameters include essential information required to perform the task. These parameters are provided by the backend when the student registers.
  - Note: A specific note ensures that the agent directly implements the LaTeX content without including placeholders or instructions.
  - Tip Section: An additional motivational section is included to encourage the agent to perform its best work.

- **Custom Tools Developed**

To augment the functionality and effectiveness of the agents within the multi-agent system, custom tools were developed the LangChain framework [18]. These tools address specific tasks that are critical to the personalized tutoring platform, enabling agents to perform specialized operations that would otherwise be outside their core capabilities. The development of these tools ensures that the agents can efficiently manage tasks such as processing educational content and storing outputs in various formats. Below, we detail two examples of such custom tools: a `PDF processing tool` and an `output storage tool`.

The first one is the PDF Processing Tool

The `PDF Processing Tool` was developed to handle the conversion and extraction of text from PDF documents, a common format for educational materials. This tool allows agents to upload, process, and retrieve content from PDFs, making it easier to integrate existing

resources into the personalized learning platform. Here is the implementation of the `PDF Processing Tool`:

```python
import os
import shutil
from pypdf import PdfReader
from langchain.tools import tool

class PDFManagementTool:

    @staticmethod
    @tool("UploadPDF")
    def upload_pdf(file_path: str) -> dict:
        """Uploads a PDF file to the server and returns a confirmation message."""
        # Assuming the PDF is uploaded to a local directory for simplicity
        if not os.path.exists('uploaded_files'):
            os.makedirs('uploaded_files')
        destination = os.path.join('uploaded_files', os.path.basename(file_path))
        shutil.move(file_path, destination)
        return {"status": "success", "file_path": destination}

    @staticmethod
    @tool("ProcessPDF")
    def process_pdf(file_path: str) -> str:
        """Processes the PDF file and returns its content."""
        try:
            # Read the PDF file
            with open(file_path, 'rb') as file:
                reader = PdfReader(file)
                text_content = ""
                for page_num in range(len(reader.pages)):
                    page = reader.pages[page_num]
                    text_content += page.extract_text() + "\n"

            return f"Extracted content from {file_path}:\n\n{text_content}"

        except Exception as e:
            return f"An error occurred while processing the PDF: {e}"

    @staticmethod
    @tool("ListPDFs")
    def list_files() -> dict:
        """Lists uploaded PDF files."""
        files = os.listdir('uploaded_files') if os.path.exists('uploaded_files') else []
        return {"files": files}

    @staticmethod
    @tool("RetrievePDFContent")
    def retrieve_file_content(file_name: str) -> str:
        """Retrieves the content of a specific PDF file."""
        file_path = os.path.join('uploaded_files', file_name)
        if os.path.exists(file_path):
            return PDFManagementTool.process_pdf(file_path)
        return f"File {file_name} not found."

    @staticmethod
    @tool("DeletePDF")
    def delete_file(file_name: str) -> dict:
        """Deletes a specific PDF file."""
        file_path = os.path.join('uploaded_files', file_name)
        if os.path.exists(file_path):
            os.remove(file_path)
            return {"status": "deleted", "file_name": file_name}
        return {"status": "file not found", "file_name": file_name}

```

Listing 7: pdf_processing_tool.py

The `PDF Processing Tool` includes methods to upload, process, list, retrieve, and delete PDF files. These capabilities enable the agents to handle educational content in PDF format seamlessly.

The second one is the Output Storage Tool

The `Output Storage Tool` was developed to manage the storage and retrieval of content generated by the agents. This tool allows agents to save their output in a structured manner, ensuring that generated educational materials, such as LaTeX documents and quiz files, are stored correctly and can be easily accessed. Here is the implementation of the `Output Storage Tool`:

```python
from langchain.tools import tool
import os
import subprocess

@tool("Store Output")
def store_output(content: str, file_path: str) -> str:
    """Store the provided content into a file specified by file_path."""
    output_dir = 'output_files'
    os.makedirs(output_dir, exist_ok=True)  # Ensure the output directory exists
    full_path = os.path.join(output_dir, file_path)

    try:
        with open(full_path, 'w') as file:
            file.write(content)
        return f"Output successfully written to {full_path}"
    except Exception as e:
        return f"Failed to write output to {full_path}: {e}"

@tool("LaTeX Compiler")
def compile_latex(file_path: str, output_file: str) -> str:
    """Compiles LaTeX content from a specified file and saves the resulting PDF to the specified output file."""
    try:
        # Construct the full path to the file in the output_files directory
        full_file_path = os.path.join('output_files', file_path)
        with open(full_file_path, "r") as tex_file:
            full_latex_content = tex_file.read()

        tex_filename = "compiled_materials.tex"
```

```
        pdf_filename = os.path.join('output_files', output_file)
        log_filename = os.path.join('output_files', "compiled_materials.log")


        with open(tex_filename, "w") as tex_file:
            tex_file.write(full_latex_content)

        try:
            # Compile the LaTeX file and redirect output to log file
            subprocess.run(
                ["pdflatex", "-output-directory=output_files", tex_filename],
                check=True,
                capture_output=True,
                text=True
            )
            # Ensure the PDF was created
            if not os.path.exists(pdf_filename):
                raise Exception("PDF compilation failed.")
            return f"PDF successfully compiled and saved to {pdf_filename}"
        except subprocess.CalledProcessError as e:
            # Capture the LaTeX error log
            with open(log_filename, "r") as log_file:
                log_content = log_file.read()
            return f"An error occurred during LaTeX compilation: {e}\nLog Output:\n{log_content}"
        finally:
            # Clean up intermediate files
            for file in [tex_filename, log_filename, os.path.join('output_files', "compiled_materials.aux"),
                         os.path.join('output_files', "compiled_materials.toc"),
                         os.path.join('output_files', "compiled_materials.out"),
                         os.path.join('output_files', "compiled_materials.tex")]:
                if os.path.exists(file):
                    os.remove(file)
    except Exception as e:
        return f"Failed to read LaTeX file: {e}"

```

Listing 8: output_storage_tool.py

The Output Storage Tool includes two key functionalities:

- Store Output: This function saves the provided content into a specified file within the `output_files` directory. It ensures the directory exists and handles any errors during the write process.
- LaTeX Compiler: This function compiles LaTeX content from a specified file and generates a PDF using MikTeX [**?**]. It handles the entire compilation process, captures any errors, and cleans up intermediate files to ensure a smooth workflow.

- **Crew Setup and Detailed Execution of Agents**

The crew setup involves defining the rules of execution for the agents and tasks, ensuring they work together to achieve the learning objectives. The main file (`crew_setup.py`) handles the setup and execution of the CrewAI agents within the system.

Crew Setup

The setup process starts by loading environment variables, creating necessary directories, and initializing agents and tasks. This ensures that the environment is prepared for the agents to operate effectively.

```
from api.crewai.agents import TutorAgents
from api.crewai.tasks import TeachingTasks
from crewai import Crew
import os
from dotenv import load_dotenv
from django.conf import settings
from api.models import User, Lesson, TaskResult, QuizResult

import threading

load_dotenv()

# Create output_files directory if it doesn't exist
if not os.path.exists('output_files'):
    os.makedirs('output_files')

class TutorCrew:
    def __init__(self, user_id, lesson_id, quiz_results=None):
        self.user = User.objects.get(id=user_id)
        self.lesson = Lesson.objects.get(id=lesson_id)

        self.student_name = self.user.username
        self.student_class = self.user.learning_class
        self.student_level = self.user.learning_level
        self.student_learning_preferences = self.user.lesson_types
        self.feedback = None
        self.existing_state = None
        self.quiz_results = quiz_results

        # Fetch lesson text or process PDF
        if self.lesson.lesson_text:
            self.lesson_content = self.lesson.lesson_text
        elif self.lesson.lesson_pdf:
            self.lesson_content = PDFManagementTool.process_pdf(self.lesson.lesson_pdf.path)

        # Instantiate your custom agents and tasks
        self.agents = TutorAgents()
        self.tasks = TeachingTasks()

        # Create instances of your agents
        self.agent_dict = {
            "learning_path_tutor": self.agents.learning_path_tutor(),
            "practical_tutor": self.agents.practical_tutor(),
            "profiler": self.agents.profiler(),
            "digital_teacher": self.agents.
```

```
    digital_teacher(),
            "communication_chatbot": self.agents
    .communication_chatbot()
        }

```

Listing 9: Setup in crew_setup.py

In this setup, the `TutorCrew` class initializes the environment, fetches user and lesson details, and prepares the agents and tasks. Agents are created using the `TutorAgents` class, and tasks are created using the `TeachingTasks` class.

Running All Tasks

The `run_all_tasks` method defines the tasks for each agent and executes them asynchronously. This ensures that each agent performs its designated task, and the overall execution is efficient and responsive.

```
def run_all_tasks(self):
    self.task_results = {}
    self.tasks_list = {
        "learning_path_tutor": self.tasks.
    adapt_learning_path(
            self.agent_dict["learning_path_tutor
    "],
            self.student_name,
            self.lesson_content,
            self.student_class,
            self.student_level,
            self.student_learning_preferences,
            self.feedback,
        ),
        "practical_tutor": self.tasks.
    apply_practical_knowledge(
            self.agent_dict["practical_tutor"],
            self.student_name,
            self.lesson_content,
            self.student_class,
            self.student_level,
            self.student_learning_preferences,
            self.feedback,
            self.quiz_results  # Pass quiz
    results to the task
        ),
        "profiler": self.tasks.profile_student(
            self.agent_dict["profiler"],
            self.student_name,
            self.lesson_content,
            self.student_class,
            self.student_level,
            self.student_learning_preferences,
            self.feedback,
        ),
        "digital_teacher": self.tasks.
    generate_report(
            self.agent_dict["digital_teacher"],
            self.student_name,
            self.student_class,
            self.student_level,
            self.student_learning_preferences,
            self.feedback,
            self.existing_state,
        ),
        "communication_chatbot": self.tasks.
    communicate_with_student(
            self.agent_dict["
    communication_chatbot"],
            self.student_name,
            self.lesson_content,
            self.student_class,
            self.student_level,
            self.student_learning_preferences,
            self.feedback,
        )
    }

    # Run tasks asynchronously
    thread = threading.Thread(target=self.
    execute_tasks)
    thread.start()

    # Return the path of the generated PDF file
    pdf_path = os.path.join('/output_files', '
    compiled_materials.pdf')
    return pdf_path

```

Listing 10: run_all_tasks in crew_setup.py

In this method, tasks are defined for each agent using the `TeachingTasks` class. The tasks are then run asynchronously using a separate thread to ensure that the execution does not block the main application flow.

Executing Tasks

The `execute_tasks` method runs the defined tasks and stores the results in the database. This method ensures that all tasks are executed in sequence and their outputs are recorded for further use.

```
def execute_tasks(self):
    # Define the crew with all agents and tasks
    crew = Crew(
        agents=list(self.agent_dict.values()),
        tasks=list(self.tasks_list.values()),
        verbose=True,
    )

    # Execute the crew's tasks and store the
    results
    crew_results = crew.kickoff()

    # Store results in the database
    for task, result in zip(self.tasks_list.keys
    (), crew_results):
        task_result = TaskResult.objects.create(
            lesson=self.lesson,
            task_type=task,
            result=result
        )
        self.task_results[task] = task_result

```

Listing 11: execute_tasks in crew_setup.py

The `execute_tasks` method creates a `Crew` instance with the defined agents and tasks, executes the tasks, and stores the results in the `TaskResult` model. This ensures that each task's output is captured and can be used for further analysis or feedback.

Asynchronous Task Execution

Specific agents can perform tasks outside the main loop, enhancing system responsiveness and adaptability. The `execute_agent_task` method allows individual agents to execute specific tasks as needed.

```
def execute_agent_task(self, agent_key, task_type, feedback):
    selected_agent_instance = self.agent_dict[agent_key]
    selected_task = None

    if agent_key == "practical_tutor":
        if task_type == "create_exercise":
            selected_task = self.tasks.apply_practical_knowledge(
                selected_agent_instance,
                self.student_name,
                self.lesson_content,
                self.student_class,
                self.student_level,
                self.student_learning_preferences,
                feedback,
                self.quiz_results
            )
        elif task_type == "correct_exercise":
            selected_task = self.tasks.correct_exercise(
                selected_agent_instance,
                self.student_name,
                self.lesson_content,
                self.student_class,
                self.student_level,
                self.student_learning_preferences,
                feedback,
                self.quiz_results  # Pass quiz results to the task
            )
    else:
        if agent_key == "learning_path_tutor":
            selected_task = self.tasks.adapt_learning_path(
                selected_agent_instance,
                self.student_name,
                self.lesson_content,
                self.student_class,
                self.student_level,
                self.student_learning_preferences,
                feedback,
            )
        elif agent_key == "profiler":
            selected_task = self.tasks.profile_student(
                selected_agent_instance,
                self.student_name,
                self.lesson_content,
                self.student_class,
                self.student_level,
                self.student_learning_preferences,
                feedback,
            )
        elif agent_key == "communication_chatbot":
            selected_task = self.tasks.communicate_with_student(
                selected_agent_instance,
                self.student_name,
                self.lesson_content,
                self.student_class,
                self.student_level,
                self.student_learning_preferences,
                feedback,
            )

    # Execute the updated task
    crew = Crew(
        agents=[selected_agent_instance],
        tasks=[selected_task],
        verbose=False,
    )
    updated_result = crew.kickoff()

    # Ensure task_type is set correctly
    if task_type is None:
        task_type = agent_key  # Default to agent_key if task_type is None

    # Store the updated result in the database
    task_result = TaskResult.objects.create(
        lesson=self.lesson,
        task_type=task_type,
        result=updated_result
    )

    # Return the updated result
    return updated_result if updated_result else "No output from task."
```

Listing 12: execute_agent_task in crew_setup.py

This method allows the system to execute specific tasks for individual agents outside the main task execution loop. This flexibility enhances the system's responsiveness and allows for real-time interactions and updates.

The orchestrated journey from registration to advanced learning modules, facilitated by our AI-driven platform, is encapsulated in the Flow Chart found in the Appendix (see Figure 24). Yellow arrows are used to represent the flow of data and communication between the various AI agents within the system. Each arrow signifies the transmission of information, such as student performance metrics, learning preferences, and feedback, facilitating a dynamic and adaptive learning environment tailored to individual needs.

5.4. Assessment

The technical deliverable focuses on the development of a personalized tutoring platform powered by autonomous AI agents. This assessment evaluates how effectively the deliverable met the outlined requirements and achieved its goals, highlighting the strengths and noting areas for improvement.

5.4.1. Backend Development.

Django Framework

The backend development utilized the Django framework to handle core functionalities such as authentication, data management, and API endpoints. The deliverable

effectively leveraged Django's features to create a robust and scalable backend. The choice of Django was justified by its comprehensive features and ease of use in developing complex web applications. This meets the requirement of developing the backend using the Django framework.

SQLite Database

The implementation of the SQLite database was seamless, providing efficient storage and retrieval of user information, learning materials, quiz results, and other relevant data. The class diagram illustrated a well-structured database schema, ensuring that data management was handled effectively. This aligns well with the requirement to build an SQLite database.

API Endpoints

API endpoints were created to handle requests from the frontend and manage interactions between the agents and the user. The use of DRF and JWT ensured secure and efficient communication between the frontend and backend. This fulfills the requirement to create secure API endpoints for managing interactions.

5.4.2. Frontend Development.

React Framework

The frontend development employed the React framework to build a user-friendly and intuitive interface. The component-based architecture facilitated the creation of reusable UI components and efficient state management. This meets the requirement to develop the frontend using the React framework.

Component Development

Various React components were developed to handle user interactions, display content, and manage state. Components such as Registration, Login, Lesson Upload, Quiz Interaction, and Chatbot Interface were well-designed and integrated seamlessly with the backend. This aligns with the requirement to develop React components for managing user interactions and displaying content.

Integration

The integration between the frontend and backend was facilitated through API calls, ensuring efficient data retrieval and updates. The frontend components effectively communicated with the backend to fetch and submit data, providing a seamless user experience. This fulfills the requirement to facilitate communication between the frontend and backend.

Responsiveness and Styling

The styling of the platform needs further refinement to ensure it is polished and adapts well to different sizes of browsers and devices. Improving the responsiveness and visual consistency across various screen sizes will enhance the user experience. This is an area that requires attention to ensure the platform meets modern web standards.

5.4.3. Multi-Agent System and NLP.

Multi-Agent System

The design and implementation of a multi-agent system using the CrewAI framework supported personalized learning experiences. Agents with specific roles, such as the Learning Path Tutor, Practical Tutor, Profiler, Digital Teacher, and Communication Chatbot, were created to manage various educational tasks. This meets the requirement to design and implement a multi-agent system.

Natural Language Processing (NLP)

Advanced language models, such as OpenAI's ChatGPT, were integrated to enable natural language interactions and content generation. This allowed for dynamic and responsive communication with students, enhancing the learning experience. This fulfills the requirement to integrate NLP for natural interactions.

PDF Processing

A tool was implemented to convert uploaded PDF lessons into text format for further processing. This functionality ensured that existing educational materials could be seamlessly integrated into the platform. However, the PDF compiler occasionally fails for unknown reasons, which requires further investigation and fixing to ensure reliability. This partially meets the requirement to implement a tool for PDF processing, with room for improvement.

Content Generation

The platform developed functionalities for generating customized learning materials in LaTeX format and compiling them into PDF documents. This allowed for the creation of personalized educational content tailored to individual student needs. This fulfills the requirement to develop functionalities for content generation.

Quiz Creation and Correction

Tasks were implemented for creating quizzes in JSON format and providing detailed feedback on quiz results. This ensured that students received immediate and personalized feedback on their performance, enhancing the learning process. This meets the requirement to implement tasks for quiz creation and correction.

Student Profiling and Feedback Analysis

The platform analyzed student preferences and adapted the learning approach accordingly. Feedback from students was used to refine educational content, ensuring continuous improvement in the learning experience. This fulfills the requirement for student profiling and feedback analysis.

Report Generation

Comprehensive reports were generated for teachers, validating content and providing insights into student performance. This functionality supported educators in monitoring and supporting student progress. This meets the requirement to generate comprehensive reports for teachers.

Chatbot Interaction
Natural language interactions with students were facilitated through a communication chatbot, providing real-time support and guidance. This enhanced student engagement and ensured responsive communication. This fulfills the requirement to facilitate natural language interactions with students.

5.4.4. Areas for Improvement.

PDF Compiler Reliability
The PDF compiler occasionally fails, which affects the seamless generation of educational materials. Addressing this issue to ensure consistent and reliable PDF compilation is crucial for the platform's effectiveness.

Autonomous Learning Progression
Currently, the system requires students to manually request new quizzes, lessons, and other learning materials. Implementing a fully autonomous system that increases the difficulty and provides new content automatically as students complete their tasks would significantly enhance the user experience and ensure continuous, uninterrupted learning.

Responsiveness and Styling
The platform's styling needs improvement to ensure it is fully polished and adapts well to different screen sizes and devices. Enhancing the responsiveness and visual consistency across various browsers and devices will improve the overall user experience. This includes refining CSS, testing across multiple devices, and ensuring a cohesive design.

5.4.5. Conclusion.

The technical deliverable successfully met the outlined requirements for the development of a personalized tutoring platform powered by autonomous AI agents. The backend development using Django, the frontend development using React, and the implementation of a multi-agent system with NLP and custom tools were effectively executed. The platform provided a robust, scalable, and user-friendly environment for personalized learning. Addressing the PDF compiler's reliability, improving styling and responsiveness, and implementing autonomous learning progression will further enhance the system's effectiveness and user experience. Overall, the deliverable achieved its goals, providing a comprehensive and well-integrated solution for personalized tutoring.

Acknowledgment

The author acknowledges the invaluable support and guidance of Doctor Sana Nouzri. Her expertise, encouragement, belief in the author's abilities, and dedicated follow-up throughout the project have been instrumental in overcoming challenges and achieving success. Her mentorship has made a significant impact on this work.

6. Conclusion

The project successfully addressed the scientific inquiry by delivering a comprehensive analysis of how autonomous AI agents can improve personalized learning within a tutoring system. By exploring the fundamentals of personalized and adaptive learning methodologies, the project highlighted the potential of agent-oriented programming (AOP) and AI in creating effective educational settings.

The scientific deliverable delved deeply into essential concepts of personalized and adaptive learning, clearly illustrating the connection between the functionalities of AI agents and enhanced learning outcomes. This foundational understanding informed the subsequent technical development, ensuring a well-grounded approach to creating the personalized tutoring platform.

The technical aspect of the project concentrated on developing a multi-agent system using the CrewAI framework, building a sturdy backend with Django and SQLite, and designing a user-friendly frontend with React. These elements were seamlessly integrated to form a dynamic educational platform capable of delivering individualized learning experiences tailored to each student's needs.

The project effectively showcased the synergy between AOP and AI in offering a flexible, scalable, and highly personalized learning environment. By harnessing the strengths of both paradigms, the platform can adjust to the unique preferences and progress of each student, thereby boosting engagement, motivation, and academic performance.

Despite the project meeting its goals, there are opportunities for further enhancement. Future work could involve incorporating a wider range of variables, such as socio-emotional learning factors, to further personalize the educational experience. Moreover, enhancing the AI algorithms to more accurately anticipate and address student needs would further increase the platform's effectiveness.

In conclusion, the project effectively demonstrated the suitability of AOP and AI for enhancing personalized learning. By fulfilling both the scientific and technical requirements, the project underscored the potential of integrating advanced AI techniques within educational systems, paving the way for future innovations in personalized tutoring.

7. Plagiarism statement

I declare that I am aware of the following facts:

- I understand that in the following statement the term "person" represents a human or **ANY AUTOMATIC GENERATION SYTEM**.
- As a student at the University of Luxembourg I must respect the rules of intellectual honesty, in particular not to resort to plagiarism, fraud or any other method that is illegal or contrary to scientific integrity.
- My report will be checked for plagiarism and if the plagiarism check is positive, an internal procedure will be started by my tutor. I am advised to request a pre-check by my tutor to avoid any issue.
- As declared in the assessment procedure of the University of Luxembourg, plagiarism is committed whenever the source of information used in an assignment, research report, paper or otherwise published/circulated piece of work is not properly acknowledged. In other words, plagiarism is the passing off as one's own the words, ideas or work of another person, without attribution to the author. The omission of such proper acknowledgement amounts to claiming authorship for the work of another person. Plagiarism is committed regardless of the language of the original work used. Plagiarism can be deliberate or accidental. Instances of plagiarism include, but are not limited to:

  1) Not putting quotation marks around a quote from another person's work
  2) Pretending to paraphrase while in fact quoting
  3) Citing incorrectly or incompletely
  4) Failing to cite the source of a quoted or paraphrased work
  5) Copying/reproducing sections of another person's work without acknowledging the source
  6) Paraphrasing another person's work without acknowledging the source
  7) Having another person write/author a work for oneself and submitting/publishing it (with permission, with or without compensation) in one's own name ('ghost-writing')
  8) Using another person's unpublished work without attribution and permission ('stealing')
  9) Presenting a piece of work as one's own that contains a high proportion of quoted/copied or paraphrased text (images, graphs, etc.), even if adequately referenced

  Auto- or self-plagiarism, that is the reproduction of (portions of a) text previously written by the author without citing that text, i.e. passing previously authored text as new, may be regarded as fraud if deemed sufficiently severe.

References

Scientific References

[1] Intelligent agent supported personalization for virtual learning environments. https://www.sciencedirect.com/science/article/abs/pii/S0167923605000989?via%3Dihub ScienceDirect.
[2] Investigating AI-Powered Tutoring Systems that Adapt to Individual Student Needs, Providing Personalized Guidance and Assessments. https://epess.net/index.php/epess/article/view/727 The Eurasia Proceedings of Educational and Social Sciences.
[3] Personalized learning through AI. https://aei.ewapublishing.org/article/7bd10dc18dfc4095b863d946366bd0ea Advances in Engineering Innovation.
[4] Individualized AI Tutor Based on Developmental Learning Networks. https://ieeexplore.ieee.org/document/8985177 IEEE Journals & Magazine.
[5] iTutor: Promoting AI Guided Knowledge Interaction in Online Learning. https://ieeexplore.ieee.org/document/9867072 IEEE Conference Publication.
[6] Development of an Autonomous, Intelligent and Adaptive E-learning System. https://ieeexplore.ieee.org/document/8756889 IEEE Conference Publication.
[7] Agent-oriented programming. https://www.sciencedirect.com/science/article/pii/0004370293900349?via%3Dihub ScienceDirect.
[8] Innovations in multi-agent systems. https://www.sciencedirect.com/science/article/abs/pii/S108480450600035X?via%3Dihub ScienceDirect.
[9] Agent programming in the cognitive era. https://link.springer.com/article/10.1007/s10458-020-09453-y Autonomous Agents and Multi-Agent Systems, Springer.
[10] An Agent-Oriented Architecture For Strategy-Based Personalized E-Learning. https://ieeexplore.ieee.org/document/9627260 IEEE Conference Publication.
[11] Personal Assistance Agent in Programming Tutoring System. https://link.springer.com/chapter/10.1007/978-3-319-19728-9_37 SpringerLink.
[12] On agent-based software engineering. https://www.sciencedirect.com/science/article/pii/S0004370299001071?via%3Dihub ScienceDirect.

Technical References

[13] Django Documentation. https://docs.djangoproject.com/en/5.0/#django-documentation Django Project.
[14] SQLite Documentation. https://sqlite.org/docs.html SQLite.
[15] React Documentation. https://react.dev/learn React.
[16] CrewAI Documentation. https://docs.crewai.com/ CrewAI.
[17] CrewAI GitHub Starter Template. https://github.com/joaomdmoura/crewAI-examples/tree/main/starter_template GitHub.
[18] Langchain Documentation. https://python.langchain.com/v0.2/docs/introduction/ Langchain.
[19] OpenAI's ChatGPT API. https://platform.openai.com/docs/overview OpenAI.
[20] MikTex Documentation. https://miktex.org/ MikTex.
[21] JSON Format Documentation. https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Objects/JSON Mozilla Developer Network.
[22] RESTful API Documentation. https://docs.github.com/en/rest/about-the-rest-api/about-the-rest-api?apiVersion=2022-11-28 GitHub.
[23] JWT Documentation. https://datatracker.ietf.org/doc/html/rfc7519 IETF.
[24] Axios Documentation. https://axios-http.com/docs/api_intro Axios.

8. Appendix

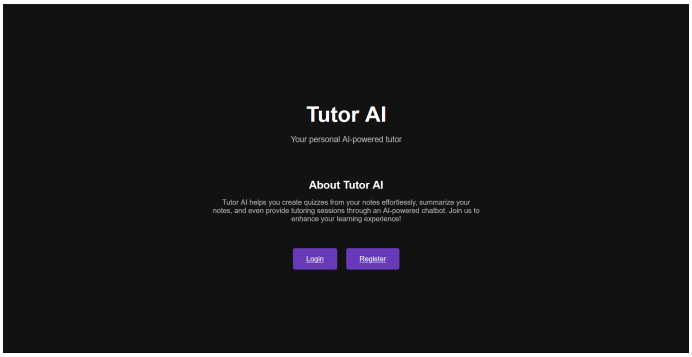


Fig. 2: Home Page describing the features and main objective of the platform.

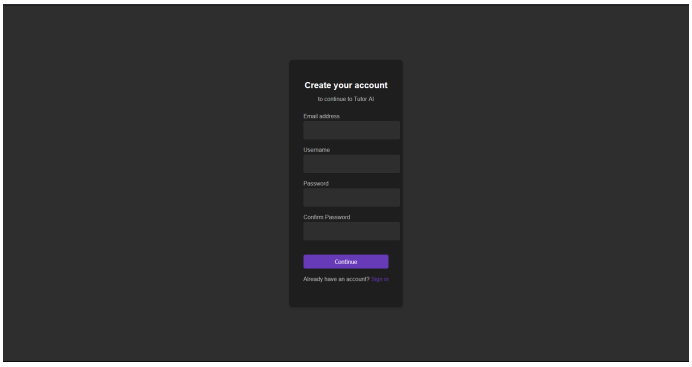


Fig. 3: Register Page Step 1 asking for user's information.

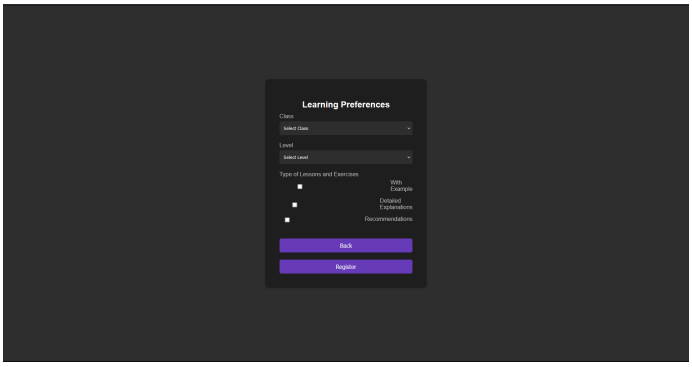


Fig. 4: Register Page Step 2 asking for user's learning preferences.

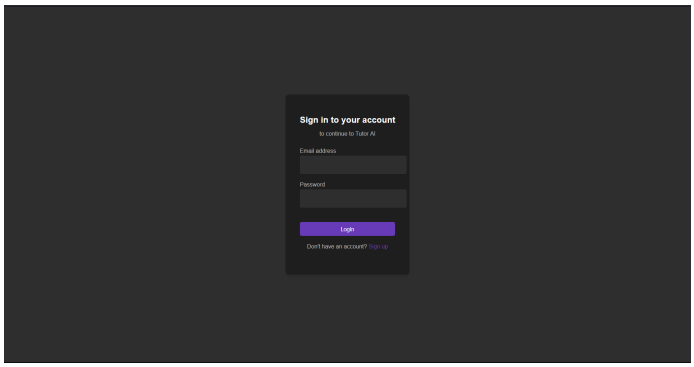


Fig. 5: Login Page.

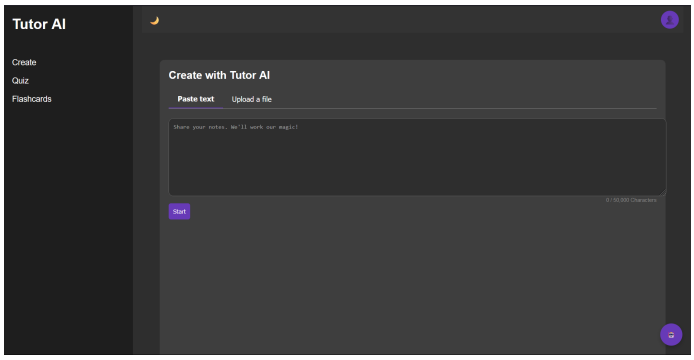


Fig. 6: Create Page with text field (Dark mode).

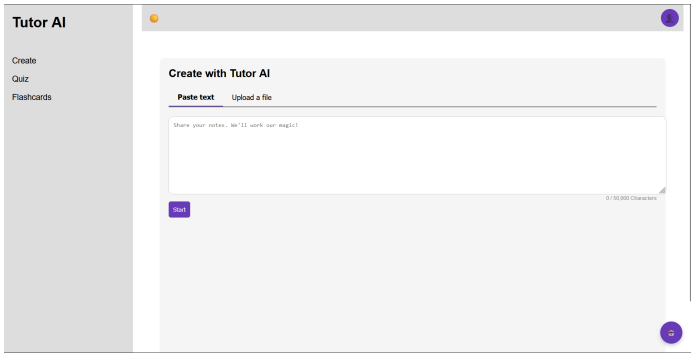


Fig. 7: Create Page with text field (Light mode).

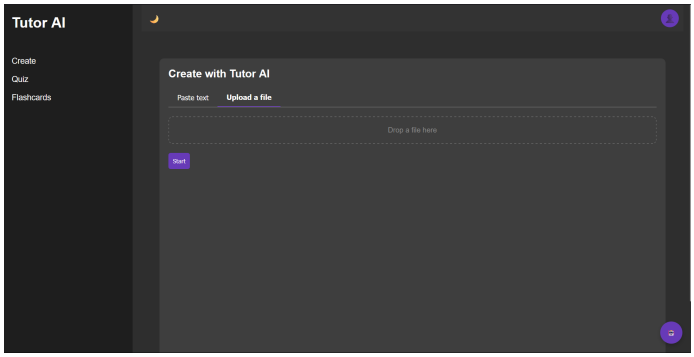


Fig. 8: Create Page with upload file field.

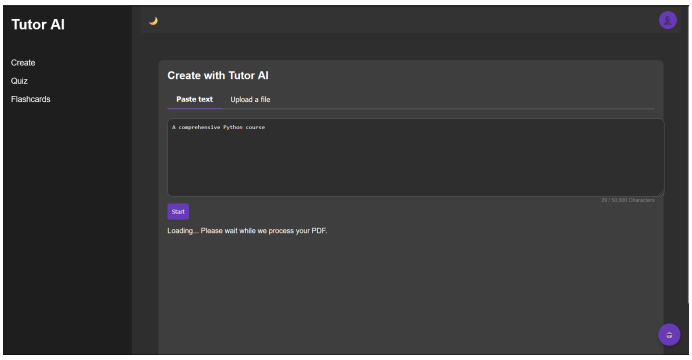


Fig. 9: Create Page Start a lesson.

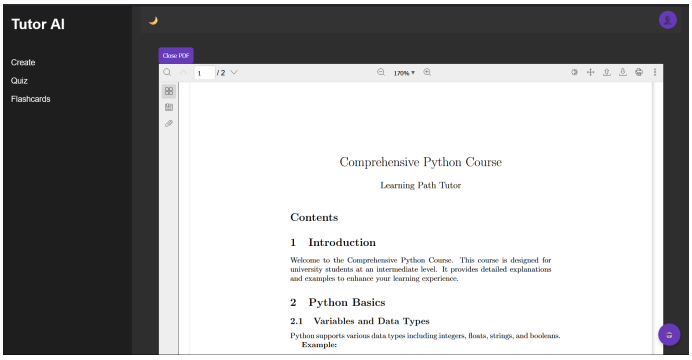


Fig. 10: Create Page Display personalized lesson.

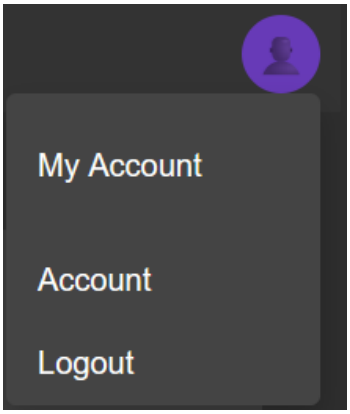


Fig. 11: Drop down menu to access the Account Page or Logout.

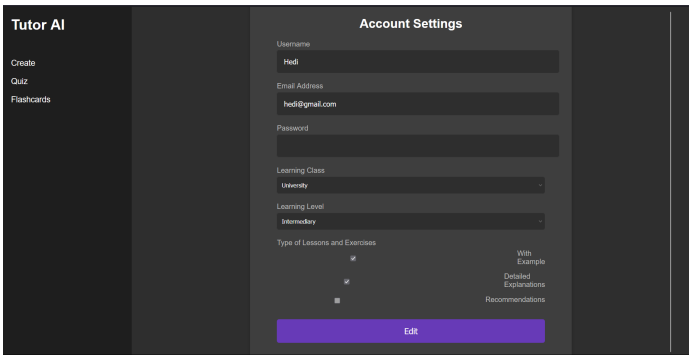


Fig. 12: Account Page in view only mode.

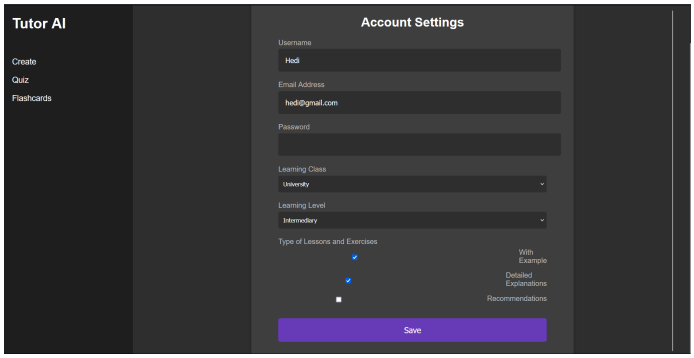


Fig. 13: Account Page in edit mode.

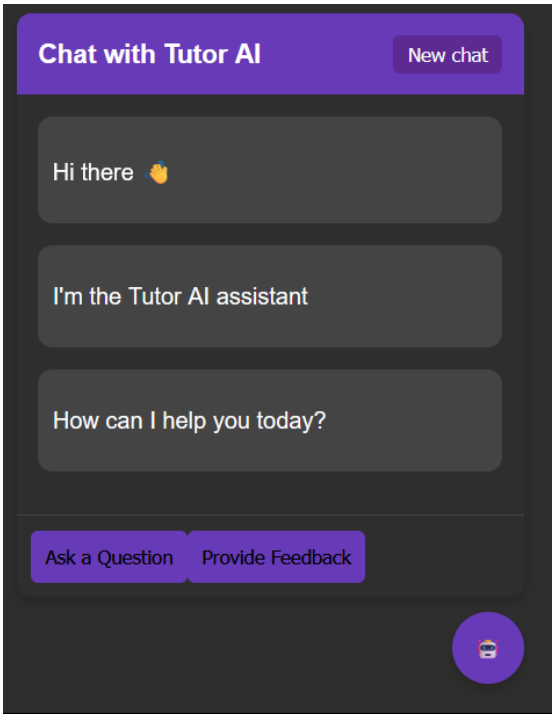


Fig. 14: Chatbot Interface with possibility the to Ask a Question or Provide Feedback.

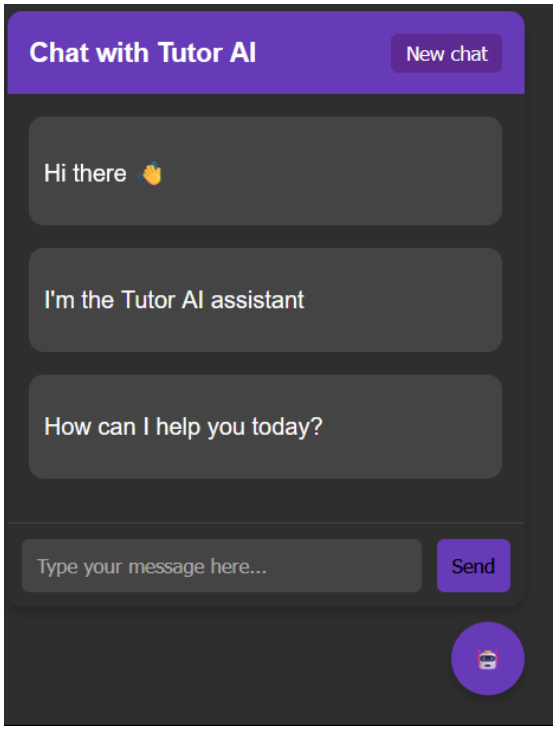


Fig. 15: Chatbot Interface after clicking on Ask a Question.

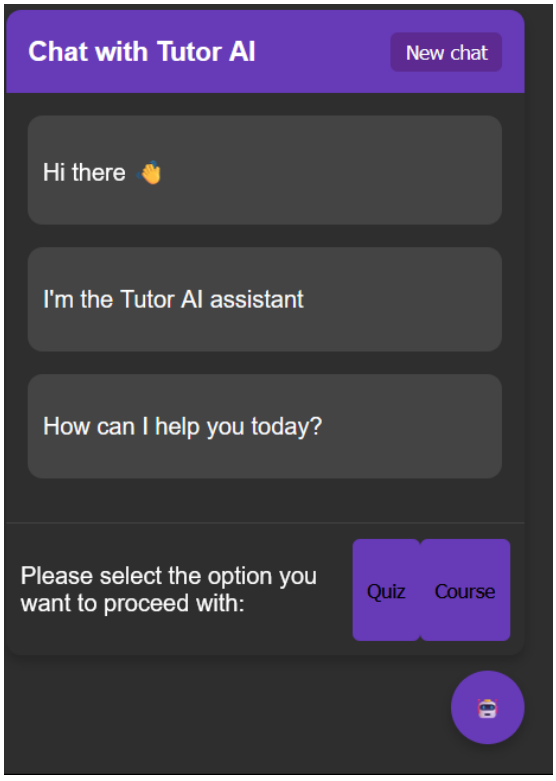


Fig. 16: Chatbot Interface after clicking on Provide Feedback.

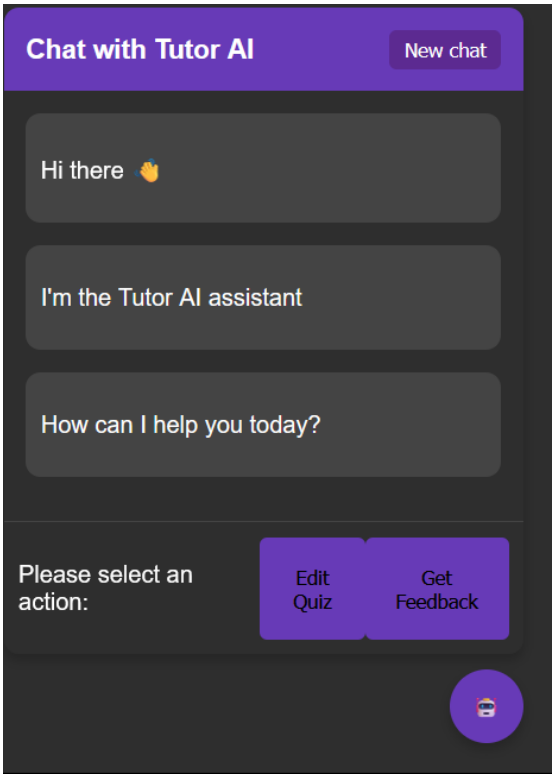


Fig. 17: Chatbot Interface after clicking on Provide Feedback - Quiz.

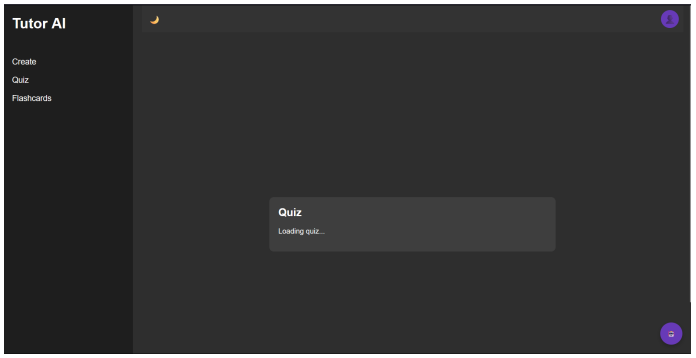


Fig. 18: Quiz Page waiting for the content to be generated.

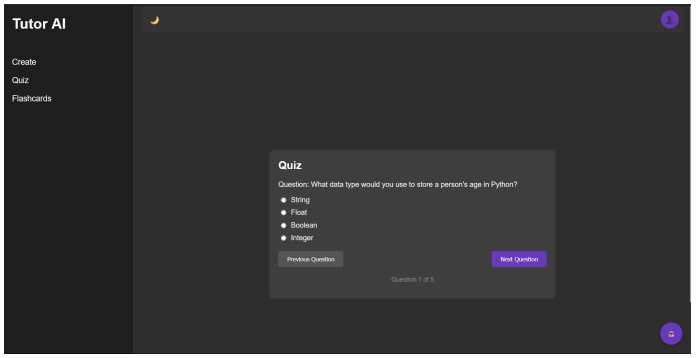


Fig. 19: Quiz Page with content available.

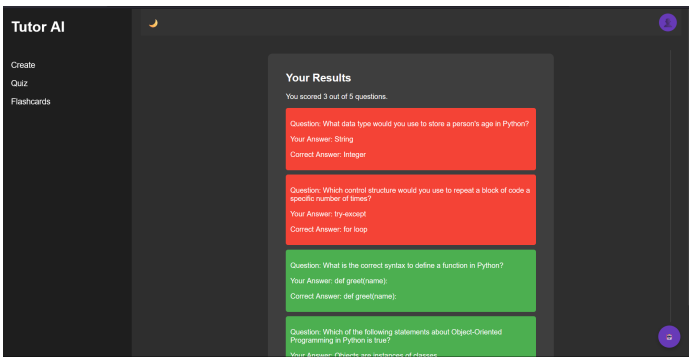


Fig. 20: Quiz Page displaying the solution.

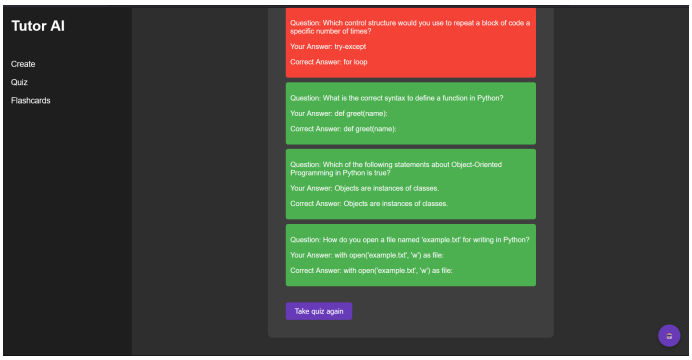


Fig. 21: Quiz Page allowing you to take the quiz again.

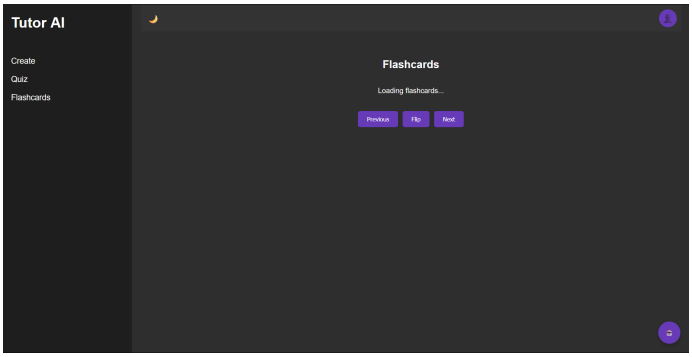


Fig. 22: Flashcards Page waiting for the content to be generated.

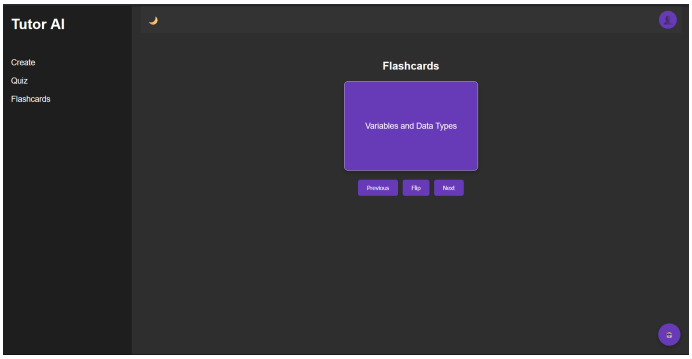


Fig. 23: Flashcards Page with content available.

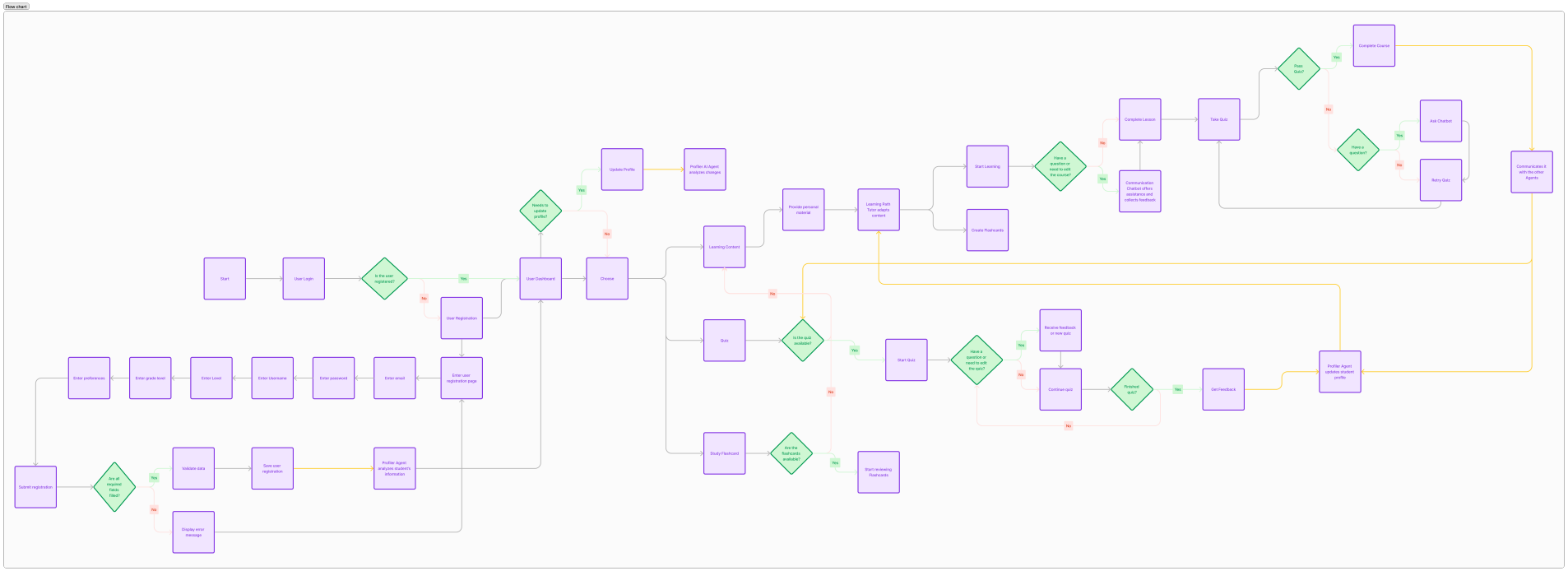


Fig. 24: Flow Chart illustrating the student journey and interactions within the platform.